

App Development Project Report

Placement Portal Application (PPA) V2

1. Student Details

Name: Nandini Parashar

Roll Number: 24F2005249

Email: 24f2005249@ds.study.iitm.ac.in

About Me: I am passionate about coding and aim to become a data scientist.

2. Project Details

Project Title: Placement Portal Application (PPA) V2

Problem Statement:

Institutes rely heavily on spreadsheets, emails, and manual coordination to manage campus recruitment, which makes tracking company approvals, placement drives, student registrations, and applications inefficient and error-prone. The goal of this project was to build a centralized, role-based Placement Portal Application that lets an Admin (institute placement cell), Companies, and Students interact with a single system, with Admin approval gating both company registration and drive creation, and Students able to discover eligible drives, apply, and track their placement history end-to-end.

Approach:

The application was built as a Flask REST API backend paired with a Vue.js 3 (CDN-based) single-page frontend, using Jinja2 only as the entry-point template that mounts the Vue app, per the assignment's constraint that Jinja2 not be used for actual UI rendering. The database schema was modelled around a single unified User table (with a role column distinguishing admin / company / student) that one-to-one extends into Company and Student profile tables, keeping authentication and role-based access control centralized. Companies create JobPositions (placement drives) that require Admin approval before becoming visible to students; Students apply to approved drives, creating Application records with a unique (student, job) constraint to prevent duplicate applications, and successful outcomes are recorded in a separate Placements table for permanent placement history. Redis was used for response caching (e.g. dashboards, drive listings) with TTL-based expiry, and Celery with Redis as the broker handles background jobs: scheduled deadline/interview reminders and monthly admin reports via Celery Beat, and an asynchronous CSV export job triggered on-demand from the student dashboard.

3. AI/LLM Declaration

I used an AI assistant (ChatGPT) during development, primarily to help scaffold SQLAlchemy model boilerplate, draft initial API route structures, and review/format documentation such as the README and this report.

The extent of AI/LLM usage is approximately 10%, limited to boilerplate generation and formatting assistance.

All core application logic — including the role-based access control design, drive-approval workflow, application/eligibility validation, caching strategy, and Celery task scheduling — along with all debugging and integration testing, was implemented and verified manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework and REST API
Flask-SQLAlchemy	ORM for programmatic SQLite schema and queries
Flask-JWT-Extended	Token-based authentication and role-based access control
Flask-CORS	Cross-origin support for the Vue.js frontend
Vue.js 3 (CDN)	Reactive single-page frontend UI
Bootstrap 5	HTML styling and responsive layout
Jinja2	Single entry-point template that mounts the Vue app
SQLite	Lightweight relational database, created programmatically
Redis	Caching layer and Celery message broker/result backend

Technology / Library	Purpose
Celery	Asynchronous background jobs and scheduled (Beat) tasks
Werkzeug	Password hashing and security utilities

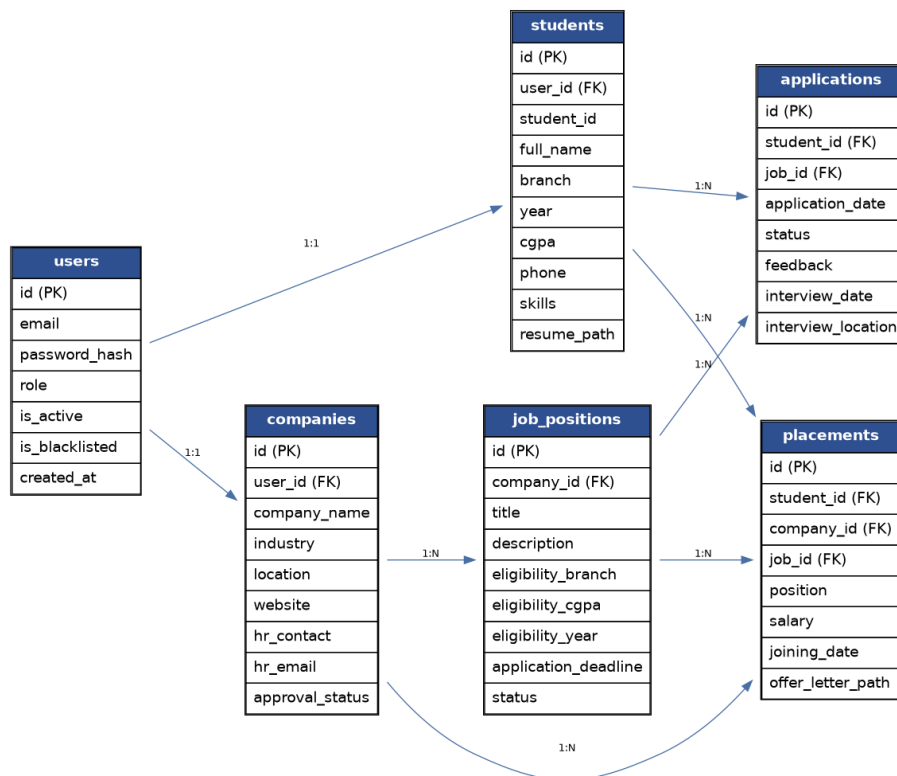
5. Database Schema / ER Diagram

The schema is centered on a unified users table, with one-to-one profile extensions for companies and students:

- users — id (PK), email, password_hash, role (admin/company/student), is_active, is_blacklisted, created_at
- companies — id (PK), user_id (FK → users.id), company_name, industry, location, website, hr_contact, hr_email, approval_status
- students — id (PK), user_id (FK → users.id), student_id, full_name, branch, year, cgpa, phone, skills, resume_path
- job_positions — id (PK), company_id (FK → companies.id), title, description, eligibility_branch, eligibility_cgpa, eligibility_year, application_deadline, status
- applications — id (PK), student_id (FK → students.id), job_id (FK → job_positions.id), application_date, status, feedback, interview_date, interview_location
- placements — id (PK), student_id (FK → students.id), company_id (FK → companies.id), job_id (FK → job_positions.id), position, salary, joining_date, offer_letter_path

Relationships:

- One-to-One → User → Company, User → Student
- One-to-Many → Company → JobPosition
- One-to-Many → JobPosition → Application, Student → Application
- One-to-Many → Student → Placement, Company → Placement, JobPosition → Placement



ER diagram generated from the actual SQLAlchemy models (models/models.py)

6. API Resource Endpoints

Authentication — /api/auth

Endpoint	Method	Description
/api/auth/login	POST	Authenticate user and issue session/JWT

Endpoint	Method	Description
/api/auth/register/student	POST	Register a new student account
/api/auth/register/company	POST	Register a new company account
/api/auth/me	GET	Get current authenticated user's profile
/api/auth/logout	POST	Clear session and log out

Admin — /api/admin

Endpoint	Method	Description
/api/admin/dashboard	GET	Overview stats: students, companies, drives
/api/admin/companies	GET	List/search all companies
/api/admin/companies/<id>/approve	PUT	Approve a pending company registration
/api/admin/companies/<id>/deactivate	PUT	Blacklist/deactivate a company
/api/admin/companies/<id>/reactivate	PUT	Reactivate a company
/api/admin/students	GET	List/search all students
/api/admin/students/<id>/deactivate	PUT	Blacklist/deactivate a student
/api/admin/students/<id>/reactivate	PUT	Reactivate a student
/api/admin/drives	GET	List all placement drives
/api/admin/drives/<id>/approve	PUT	Approve a placement drive
/api/admin/applications	GET	View all student applications
/api/admin/reports/stats	GET	Placement statistics for reporting

Company — /api/company

Endpoint	Method	Description
/api/company/dashboard	GET	Company details, drives, applicant counts
/api/company/profile	GET / PUT	View or update company profile
/api/company/drives	GET / POST	List own drives / create a new drive
/api/company/drives/<id>/status	PUT	Update a drive's status
/api/company/drives/<id>/applications	GET	View applications for a specific drive
/api/company/applications/<id>/status	PUT	Shortlist / update an application's status
/api/company/students/<id>	GET	View a student applicant's profile

Student — /api/student

Endpoint	Method	Description
/api/student/dashboard	GET	Approved drives, eligibility filters
/api/student/profile	GET / PUT	View or update student profile
/api/student/profile/resume	POST	Upload resume file
/api/student/drives	GET	Browse eligible/approved drives
/api/student/drives/<id>/apply	POST	Apply to a placement drive
/api/student/applications	GET	View own application statuses
/api/student/placements	GET	View confirmed placements
/api/student/history	GET	Full placement history
/api/student/export/csv	POST	Trigger async CSV export (Celery)
/api/student/export/status/<task_id>	GET	Poll export task status
/api/student/export/download/<filename>	GET	Download the generated CSV

A YAML API definition file (*api.yaml*) is included in the submission ZIP.

7. Background Jobs

Scheduled Jobs (Celery Beat)

- Daily deadline reminders — runs daily at 09:00 IST; emails students with drives whose application deadline falls within the next 24 hours, and posts a summary to Google Chat via webhook.
- Daily interview reminders — runs daily at 08:30 IST; emails students with interviews scheduled in the next 24 hours.
- Monthly admin report — runs on the 1st of every month at 08:00 IST; generates an HTML report (drives conducted, applications received, students selected, placements confirmed) for the previous month and emails it to the admin.

User-Triggered Async Job

- CSV export — a student triggers this from their dashboard; a Celery task compiles their full application history (Student ID, Company Name, Drive Title, Status, Dates) into a CSV, and the student is notified/can poll for completion and download the file.

8. Caching

Redis-backed caching (`cache_get` / `cache_set` / `cache_delete` helpers) is applied to frequently-read, expensive endpoints such as dashboards and drive listings, using a configurable default TTL (`CACHE_DEFAULT_TTL`) so cached data expires automatically and stays reasonably fresh.

9. Architecture

- `app.py` — Flask application factory and entry point
- `config.py` — environment-driven configuration
- `init_db.py` — programmatic database creation and default admin seeding
- `models/` — SQLAlchemy models (users, companies, students, job_positions, applications, placements)
- `routes/` — Flask blueprints: auth, admin, company, student
- `tasks/` — Celery tasks: `scheduled_tasks.py` (reminders, monthly report), `export_tasks.py` (CSV export)
- `utils/` — `cache.py` (Redis helpers), `notifications.py` (email/Google Chat), `auth_helpers.py`
- `templates/index.html` — Jinja2 entry point mounting the Vue app
- `static/js/` — Vue app and role-specific components (`admin.js`, `company.js`, `student.js`)
- `static/css/` — custom styling on top of Bootstrap 5
- `celery_worker.py` — Celery app configuration and Beat schedule

10. Video Presentation

Drive Link: https://drive.google.com/file/d/1IY1eAPsAdUToI0m0Y8Y7WaOP2hXXAfGe/view?usp=drive_link